

■ CLOUD COMPUTING · DEVOPS

\$ git init knowledge

Git, GitHub & CI/CD Fundamentals

Lec by Imran Sayyed

2.5 hour session

For first-time learners

Where this session takes you

01

Version Control

Why teams need history and a single source of truth

02

Git Basics

Repositories, commits, and the everyday commands

03

GitHub

Cloning, pushing, pulling, and shared repositories

04

Branching

Working in parallel and merging changes safely

05

CI/CD

Automating build, test, and deployment pipelines

06

GitHub Actions

Triggers, workflows, jobs, and steps in practice

Three developers, one file

Developer 1

edits login.js

Developer 2

edits login.js

Developer 3

edits login.js



same file • same time

Overwrites • lost work • conflicts

Why version control exists

1 Track history
See every change and who made it

2 Work together
Many people, one codebase, no chaos

3 Undo mistakes
Roll back to any working version

4 Safe backup
Code lives beyond one laptop

Git is a distributed version control system

Every developer keeps a full copy of the project and its entire history — right on their own machine.

// facts

Created in 2005

by Linus Torvalds

Free & open source

runs entirely offline

Industry standard

used by teams everywhere

A complete history of your code

Each saved snapshot is kept forever and stays reachable.



The three areas of Git

where you edit

Working Directory

Your actual project files



what's ready

Staging Area

Changes picked for the next snapshot

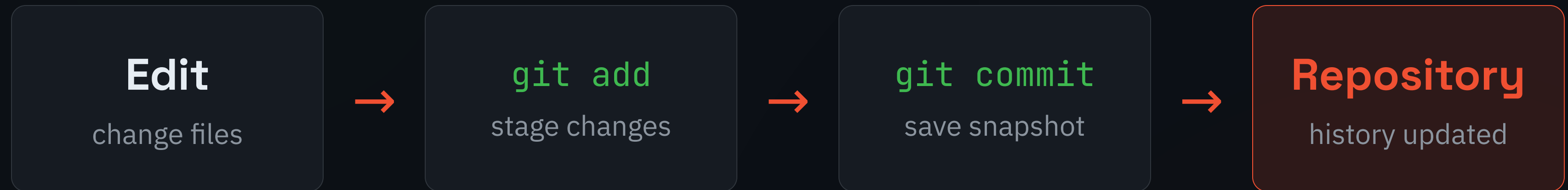


saved forever

Repository

The committed history

The everyday Git workflow



edit → git add → git commit → repository

A commit is a saved snapshot

It records exactly what your project looked like at one moment in time.

Every commit carries an author, a timestamp, and a short message.

● commit a1b2c3d

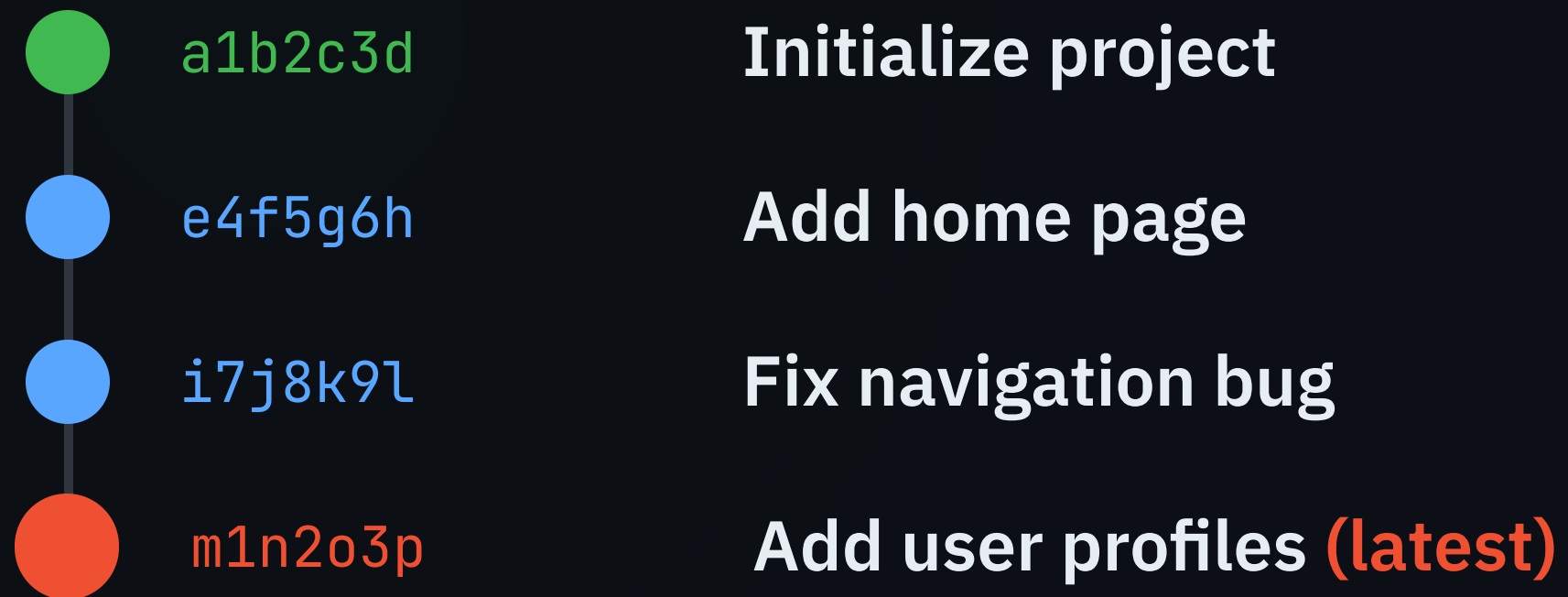
"Add login validation"

author: maya

date: Tue 14:32

2 files changed

Commits form a timeline



GitHub hosts your Git repositories online

It is a cloud home for your code where teams share, review, and collaborate from anywhere.

// github gives you

Remote storage

code backed up in the cloud

Collaboration

pull requests & reviews

Automation

CI/CD with Actions

Git is not GitHub

the tool

Git

Software on your computer

Tracks versions locally

Works fully offline

the platform

GitHub

A website in the cloud

Hosts Git repositories

Needs an internet connection

A repository is your project's home

It holds every file plus the full history of changes.

● `my-first-project` public

`README.md`

Add project intro

`index.html`

Build home page

`style.css`

Add styling

Clone copies a repo to your machine

GitHub · cloud

Remote repo

The shared online copy

`git clone`



your laptop

Local copy

Full project + history

```
git clone https://github.com/user/repo.git
```

■ P U S H

Push sends your commits up



■ P U L L

Pull brings teammates' work down



The commands you use daily

`git init` Start a new repository

`git status` See what changed

`git add` Stage your changes

`git commit` Save a snapshot

`git push` Upload to GitHub

`git pull` Download updates

■ MORE COMMANDS

A few more worth knowing

`git log`

View commit history

`git diff`

Compare changes

`git branch`

List or create branches

`git
checkout`

Switch branches

`git merge`

Combine branches

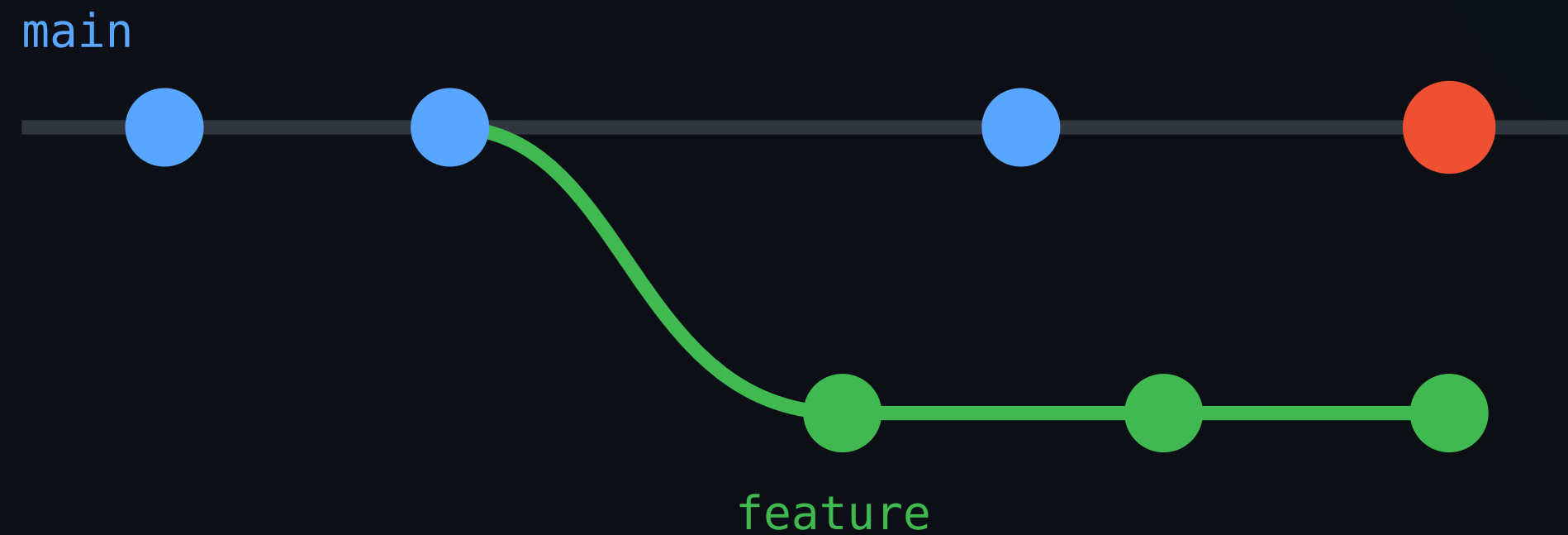
`git clone`

Copy a remote repo

■ BRANCHING

A branch is a separate line of work

Build a feature without touching the main code.



Why teams branch

A

Stay safe

Main stays stable while you experiment

B

Work in parallel

Many features built at the same time

C

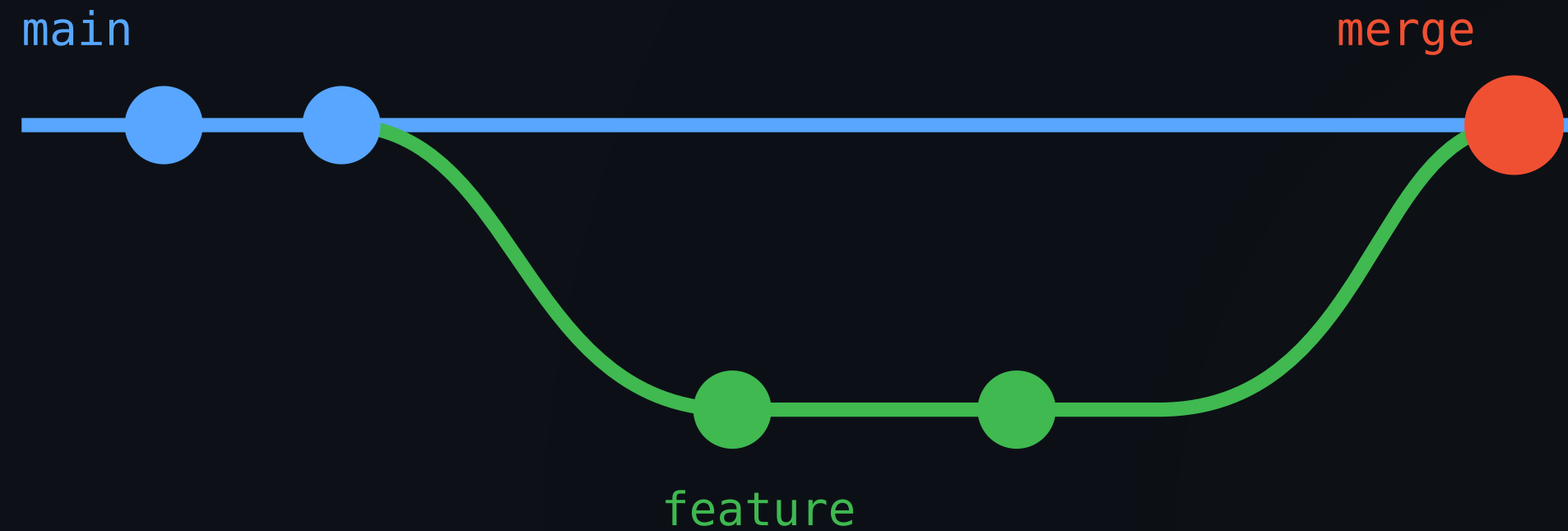
Review first

Changes are checked before merging

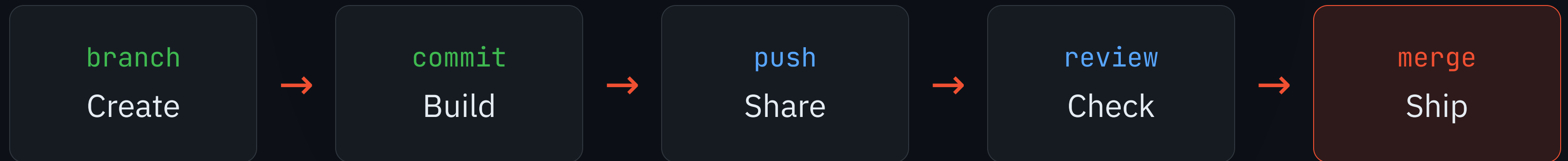
■ M E R G E

Merging brings work back together

The finished feature joins the main branch.



How a team ships a feature



Every feature follows the same safe, repeatable path.

The GitHub repository page

● octo-team / payments-api

Code Issues Pull requests Actions Settings

src/

Refactor handlers

README.md

Update docs

package.json

Bump version

Creating a new repository

New repository

Repository name

my-first-project

☒ Public ☐ Private

Create repository

Start a repository locally

● ● ● bash

```
$ mkdir my-first-project
```

```
$ cd my-first-project
```

```
$ git init
```

```
Initialized empty Git repository
```

Clone an existing repository

● ● ● bash

```
$ git clone https://github.com/octo-team/payments-api.git
```

```
Cloning into 'payments-api'...
```

```
remote: Enumerating objects: 128, done.
```

```
Receiving objects: 100% (128/128), done.
```

```
$ cd payments-api
```

Stage and commit your work

● ● ● bash

```
$ git add .
```

```
$ git commit -m "Add home page"
```

```
[main a1b2c3d] Add home page
```

```
2 files changed, 34 insertions(+)
```

Push your code to GitHub

● ● ● bash

```
$ git push origin main
```

```
Enumerating objects: 5, done.
```

```
Writing objects: 100% (3/3), done.
```

```
To github.com:octo-team/payments-api.git
```

```
a1b2c3d..e4f5g6h main → main
```

DevOps unites building and running

A culture of fast, reliable, automated delivery.



Manual releases are slow and risky

without CI/CD

Manual testing, easy to skip

Bugs found late

Stressful, rare releases

with CI/CD

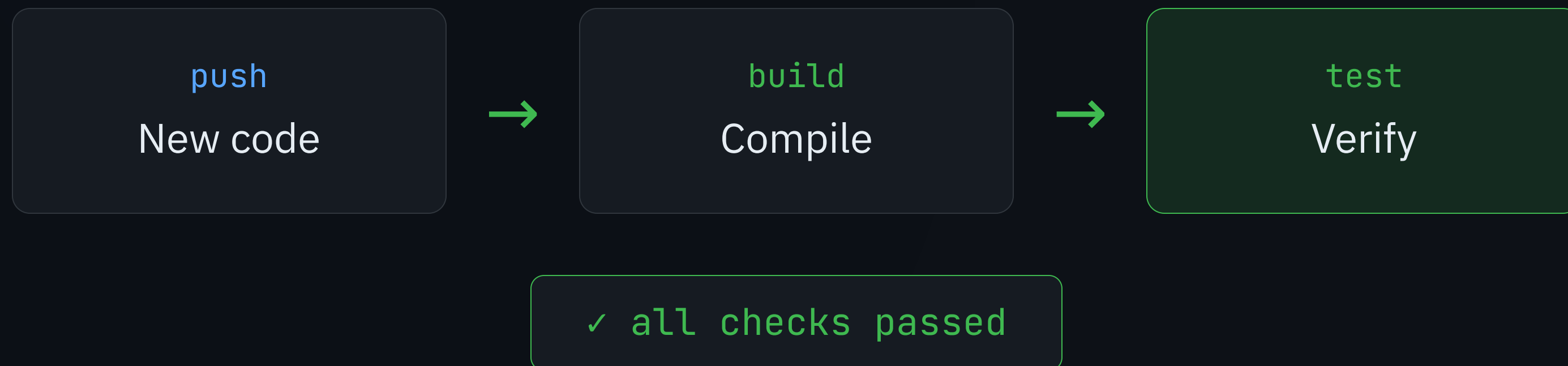
Tests run automatically

Bugs caught early

Calm, frequent releases

CI: merge and test often

Every push is automatically built and tested.

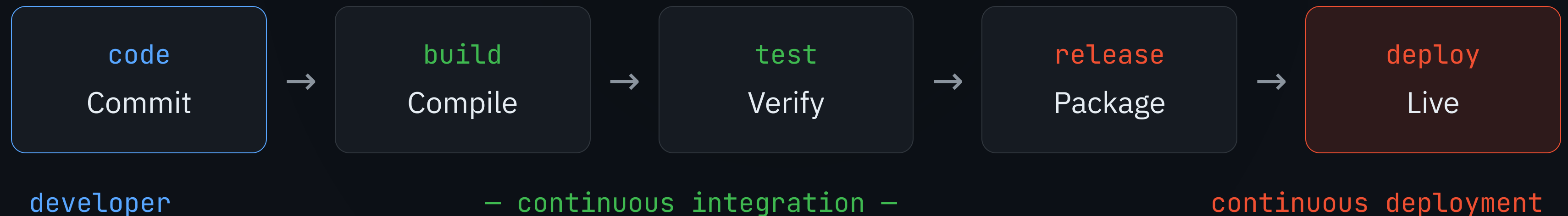


CD: ship automatically

Once tests pass, code goes live without manual steps.



The full CI/CD pipeline



GitHub Actions runs CI/CD inside GitHub

Define automated workflows that react to events in your repository — no separate server needed.

// it can

Run tests

on every push

Build apps

automatically

Deploy

to production

How a workflow is structured

Workflow · runs on an event

Job · runs on a machine

Step: checkout

Step: install

Step: test

A workflow contains jobs, and each job runs a list of steps.

Workflows are written in YAML

A simple text format of keys, values, and indentation.

```
name: CI
on: push
jobs:
  build:
    runs-on: ubuntu
```

Key: value

a setting and its value

Indentation

shows what belongs inside what

Readable

easy for humans to follow

A complete example workflow

```
.github/workflows/ci.yml
```

```
name: Run Tests
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm test
```

Watching a workflow run

Run Tests #42

success

✓ Set up job

✓ Checkout code

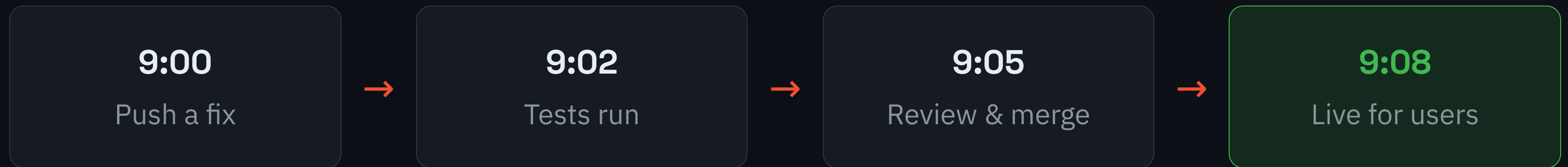
✓ npm install

✓ npm test – 24 passing

■ REAL WORLD

A day in a real team

From a developer's keyboard to live users in minutes.



Common beginner mistakes

forgetting to pull

Out-of-date code

Always pull before you start working

vague messages

"fixed stuff"

Write clear, specific commit messages

committing secrets

Leaked passwords

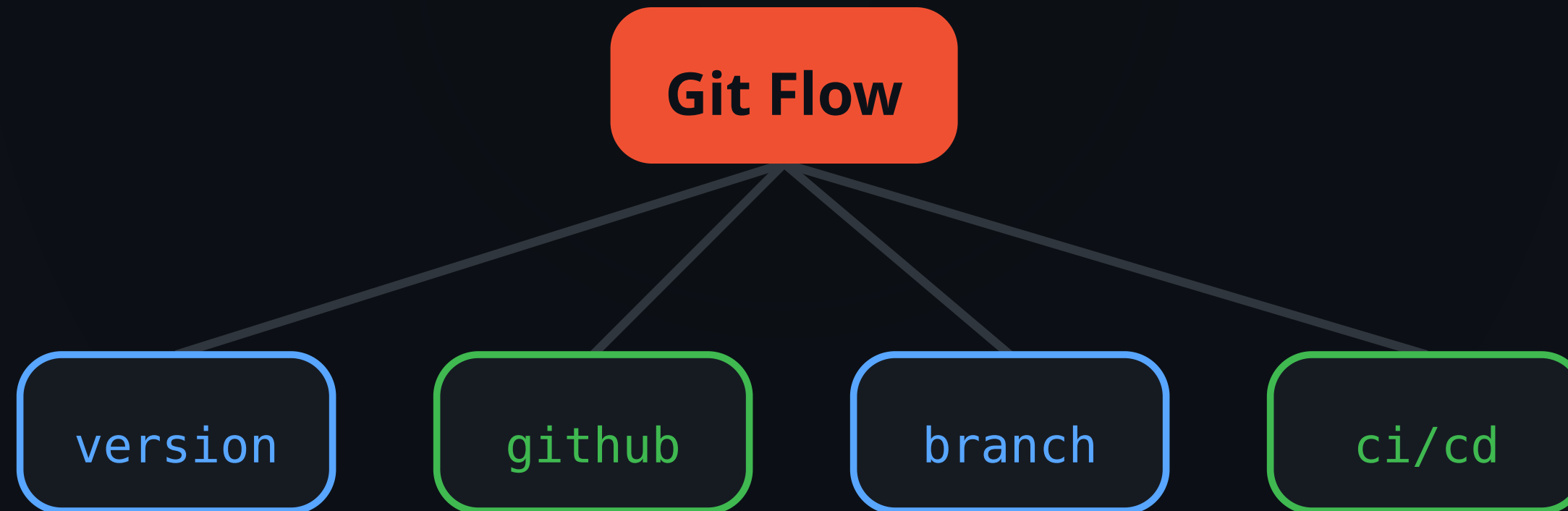
Keep keys out of the repository

working on main

Risky changes

Use a branch for every new feature

The big picture



Test yourself

Q1

Which command uploads commits to GitHub?

Q2

What is the difference between Git and GitHub?

Q3

Why do teams work in branches?

Q4

What does CI stand for?

Q5

When does a workflow run?

Q6

Name one common Git mistake.

What you now know

✓ Git tracks versions of your code

✓ GitHub hosts and shares repositories

✓ Branches let teams work in parallel

✓ CI/CD automates testing and deploys

✓ GitHub Actions runs your pipelines

✓ You can create, commit, and push

■ T H A N K Y O U

```
$ git push origin your-career
```

Questions & Discussion

Now go create a repository, make your first commit, and push it to GitHub.

```
git init
```

```
git commit
```

```
git push
```